UNSW SYDNEY

The University of New South Wales

School of Computer Science and Engineering

COMP9417 Machine Learning

Customer Feedback Classification using Machine Learning

Introduction

This project aimed to develop a robust multiclass classification model capable of categorising customer comments into one of 28 product categories. Using a dataset containing 10,000 comments, each represented by 300 features, the model was designed with a focus on accuracy and efficiency to help streamline the company's feedback management process. During development, we encountered several challenges including an imbalanced dataset, low variance across features and large feature space. Addressing these challenges was essential for us to create a model for our task.

Literature Review

Natural Language Processing (NLP) is a technique that uses machine learning to help computers understand human language (Stryker & Holdsworth 2024). Its ability to extract important information from data has led to its widespread adoption in enterprises, where the technique is used to streamline and automate business operations (Stryker & Holdsworth 2024). Most of the pre-processing methods in literature focus on data transformation, filtering, tokenisation, and encoding are used to convert the raw data into a structured format suitable for modelling. However, as our dataset was already encoded, we did not need to perform these steps. Regarding classification methods, as noted by Noori (2021), Support Vector Machines (SVM), which separates classes by maximising the margin between data points of different classes, and Naive Bayes (NB), which applies the Bayes Theorem, were found to be effective for categorising customer feedback. Furthermore, Random Forest Classifiers, which use uncorrelated decision trees to make predictions, was expected to perform well but resulted in low accuracy and precision across almost all classes (Mannemala 2021). Instead, Mannemala (2021) found that Gradient Boosting achieved the highest accuracy when classifying customer feedback, likely because it improves prediction accuracy by learning from past errors.

Methodology

Data Cleaning

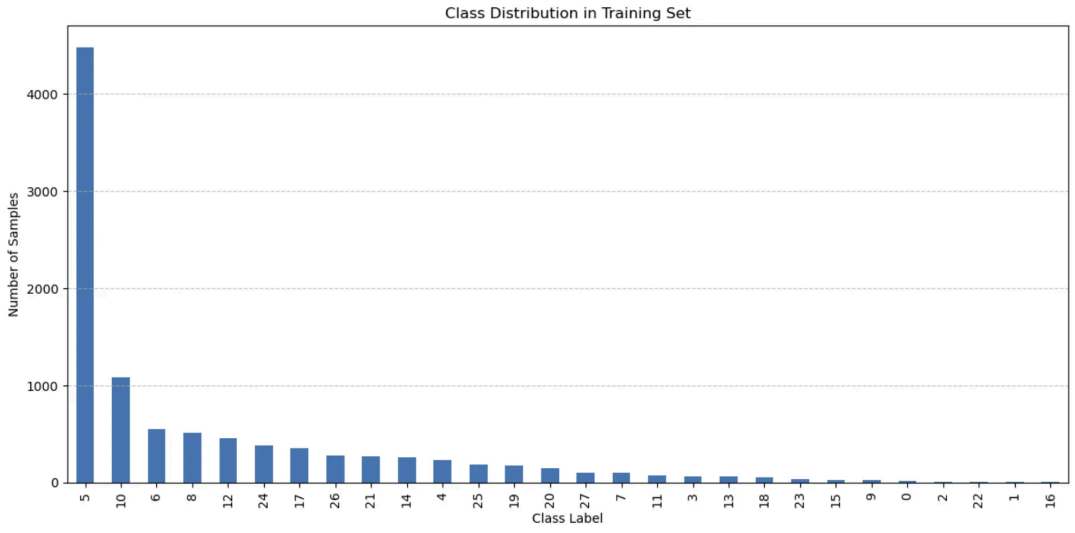
First we assessed the dataset for missing values, duplicates, and inconsistencies, and found none. While some outliers were detected using the Z-score method, they were spread across different features, with less than 0.4% of the data being affected. Since the impact of the outliers on the dataset was minimal, we chose not to remove any rows of data. Next, we observed the mean and variance of the features, finding that most features had a mean of 0 and a low variance. Low variability suggests that some features in our dataset are uninformative, and therefore feature selection must be performed prior to modelling.

Feature Correlation

To determine feature correlation, we computed a correlation matrix and visualised it using a heatmap (see Appendix A). In the heatmap, we found no solid dark color between different features, indicating that the features do not have any strong relationships. This was confirmed by examining the values in the correlation matrix (see Appendix A) where the most correlated feature pair had a correlation of 0.678, which was not strong enough to warrant removal.

Distribution of Targets

As shown in the graph below, class 5 appeared over 4000 times, whilst most other classes appeared fewer than 500 times, highlighting the significant class imbalance present in the dataset. The imbalance negatively affects model learning by introducing bias towards the majority class, making it harder for the model to predict the minority classes. As a result, the model may struggle to generalise, leading to poor performance on test data. To account for this, we can adjust the frequency of each class in the training set through oversampling and undersampling, ensuring that all classes appear an equal number of times (Data Professor 2022). Alternatively, given the large volume of data, it may be more feasible to adjust the



class weights to give more importance to underrepresented classes (Data Professor 2022). Furthermore, we must select an evaluation metric that is suitable for imbalanced datasets to ensure an accurate assessment of the model's performance across all classes.

Figure 1: Class distribution in the training set

Evaluation Metrics

The F1 score is a commonly used evaluation metric that is derived from the harmonic mean of precision and recall (GeeksforGeeks 2025). Since the F1 score combines precision and recall, it provides a

balanced measure of model performance, making it reliable for imbalanced datasets (GeeksforGeeks 2025).
Weighted cross entropy loss calculates how well the predicted class distribution matches the true class distribution (GeeksforGeeks 2024). It is calculated by applying class specific weights to the cross-entropy loss, with a lower value indicating better model performance (GeeksforGeeks 2024). This technique emphasizes underrepresented classes, making it useful with imbalanced datasets.

Hyper Parameter Tuning

Each hyperparameter plays a significant role in a model. Below are the hyperparameters that plan to tune throughout our model development process.

- class_weights: adjusts the weight for each class to handle class imbalances
- penalty, C (Logistic Regression): controls the strength and type of regularization to prevent overfitting
- n_estimators, max_depth, min_samples_leaf (XGBoost): sets model complexity to balance bias and variance
- eta/learning_rate, gamma, subsample (XGBoost): helps with the model with generalization
- early_stopping_rounds, reg_lambda: helps control overfitting of the model

Feature Extraction

Given the dataset's 300 features, we performed feature selection to remove redundant and noisy features. To begin, we applied Permutation Importance using Logistic Regression, which proved to be effective as it captures the impact of each feature on predictive performance and accounts for interactions between features. We also applied Permutation Importance with Random Forest and XGBoost but as the weighted log loss only improved for the Random Forest model. Ultimately, a combination of original XGBoost, Permutation Importance with both Logistic Regression and Random Forest were used to find these important features. When testing our Permutation Importance Logistic Regression, we found that with around 50 features, the weighted log loss was minimised, which is an indication of its optimal cutoff point. By using this insight, we selected the top 50 features from each of the three models, and took the union of these feature sets to form the final pool of informative features for downstream modelling.

Model Development

During development, we ensured that all models were tuned using 3-fold stratified cross-validation and we were tuning hyperparameters. First we began individually testing simple models such as Regression, Random Forest and XGBoost. However, the results were poor, especially for minority classes, where the weighted log loss revealed that the model was overlooking minority classes. To account for this, we used models with inbuilt mechanisms for handling class imbalances by assigning class weights as being inversely proportional to the class frequency. Additionally, the poor results may have been due to the variability in scale across the features, and so we decided to apply a StandardScaler to ensure that all features were treated with equal importance. After this, we tested more complex models like Light GBM and SVM. However, we found that the LightGMB model produced unstable performance on minority classes even after applying class weights whereas, SVM ws computationally inefficient and scaled poorly with large datasets. Initially, we considered individual models, but after further research, we opted for a stacking ensemble method that combined the strengths of multiple models. We then experimented with various 2-way and 3-way stacking configurations, ultimately finding that a 3-way stack of Logistic Regression, Random Forest and XGBoost performed the best.

Results

Evaluation of design choice

Stacking	Logistic Regression	Random Forest	XGBoost	Weighted Log Loss on Train Set	Weighted Log Loss on Test
Logistic Regression	Yes	No	No	0.0052	0.0058
Random Forest	No	Yes	No	0.0013	0.0202
XGBoost	No	No	Yes	6.5139	0.0092

#number	Logistic Regression Parameters	Random Forest Parameters	XGBoost Parameters	Weighted Log Loss on Training Set (6dp)	Weighted Log Loss on Test Set (6dp)
1	max_iter=1000, class_weight='balanced'	n_estimators=100, class_weight='balanced', random_state=42	n_estimators=100, eval_metric='mlogloss', random_state=42	**0.003671**	**0.004300**
2	max_iter=1000, penalty='l2', class_weight=None	n_estimators=100, max_depth=None, min_samples_leaf=1	eta=0.3, gamma=0, max_depth=15, subsample=1	0.003130	0.004712
3	C=7.23, solver='liblinear', class_weight='balanced'	n_estimators=333, max_depth=4, min_samples_split=25, class_weight='balanced'	max_depth=2, eta=0.15, colsample_bytree=0.2, random_state=0	0.003784	0.004447
4	max_iter=1000, class_weight=None	n_estimators=100, class_weight=None, random_state=42	n_estimators=100, eval_metric='mlogloss', random_state=42	0.003517	0.004705
5	penalty='none', class_weight='balanced'	bootstrap=False, n_estimators=300, max_features=None	early_stopping_rounds=None, reg_lambda=0, learning_rate=0.5	30 min + No result	30 min + No result

Logistic Regression + Random Forest	Yes	Yes	No	0.0032	0.0045
Two-way stacking	Yes	Yes	No	0.0032	0.0045
Two-way stacking	No	Yes	Yes	0.0034	0.0050
Three-way stacking	Yes	Yes	Yes	0.0037	0.0043

Table 1: Table of Performance based on different models

To find the final model, we first employed XGBoost, Random Forest, and Logistic Regression independently, and then tested these models in different stacks. Upon looking at Table 1, although some of the models or stacks have relatively lower weighted log losses on the training set, they usually come with the downfall of performing relatively poorly on the test set. In the end, to ensure better generalisation of the model, the three-way stacking of Logistic Regression, Random Forest, and XGBoost was chosen.

Hyperparameter Evaluation

Table 2: Hyperparameter Testing on different Models

As shown in the table below, the combination of hyperparameters had varying effects on the weighted log loss of the training and test sets. Set 1, representing the hyperparameters selected for our final model, was the most effective as it had the best weighted log loss across both the training and test sets. While Set 2 had a lower training weighted log loss, the model with these hyperparameters showed signs

of overfitting, as the test set had a higher test weighted log loss. The other hyperparameter combinations that we tested had both a higher training and test weighted log loss, leading us to conclude that the hyperparameters chosen for our final model were optimal.

Analysis of important features

Based on our EDA, we found that tree-based models like Random Forest and XGBoost were the most effective methods to identify important features, due to their ability to handle skewed data. For simplicity, Random Forests were used as a starting point, however, due to its inability to capture nonlinear interactions, it was unable to be used alone. Logistic Regression was then used for its interpretability and high performance, but it lacked non-linear learning capabilities. When combining these two to improve feature detection, it missed gradient-based refinement, and therefore XGBoost was introduced to improve the algorithm and its sensitivity to prediction. Ultimately, a combination of the 3 models gave the best feature set, identifying 105 distinct features (see Appendix B). These capture linear and non-linear relationships and improve the feature being robust. The feature confirmed consistency across all the modelling techniques as well.

Discussion

Comparison of methods, features and performance

During feature selection, we tested different combinations of three models – Logistic Regression, Random Forest, and XGBoost. Due to their learning biases, each model prioritized different features. For example, tree-based models overemphasised certain patterns, while logistic regression missed nonlinear interactions. As a result, feature subsets were skewed or incomplete, which led to decreased performance when used separately. To address this, we took the union of the top-ranked features from each of the three approaches to capture both linear and non-linear relationships. This ensemble approach enhances model accuracy and features robustness across various classifiers.

As shown in table 2, five different hyperparameters were tested to evaluate their effect on model performance. Set 1, our baseline, incorporated a balanced class weight and moderate complexity. It performed the best, with the low-weighted training and test log loss and the smallest difference between the two results. Set 2 removed regularisation to test a complex model but this resulted in overfitting. In Set 3, we applied more noise-tuned parameters but it underperformed Set 1, suggesting non-optimal tuning. Set 4 completely ignored the balance class which as expected performed poorly. For Set 5, we removed regularisation, incorporated early stopping and increased model size. However, the computational performance was too long, as a result, so it was not selected.

When choosing a model for classification, different combinations of three models, Logistic Regression, Random Forest and XGBoost, were considered. In reference to Table 1, Random Forest had the lowest weighted log loss on the training set, but had a relatively high weighted log loss on the test set. This is an indication of overfitting, and therefore was not chosen to exclusively be the final model. We can also see in the table above that both Logistic Regression and XGBoost independently as models also perform poorly in comparison to our final three-way stacking method. We can also see that even though both the two-way stacks have a lower weighted log loss on the training sets, they perform worse than the final model, which better ensures generalisation.

Metric choice

While both weighted log loss and F1 can handle imbalanced datasets, weighted log loss is more suited for this task because it addresses class imbalances more effectively. It does so by applying class weights which penalises the misclassification of minority classes more heavily than F1, resulting in better calibrated outputs. In contrast, the F1 score can mask the performance of underrepresented classes. Therefore, we chose weighted-log loss as our metric.

Future Improvements

Machine learning models learn the patterns and relationships present in the source distribution, so when exposed to data with a different distribution or relationships, their performance can degrade. The problem in ‘Test Set 2’ is consistent with a covariate shift, which is a change in the density distribution of the inputs. By comparing the descriptive statistics of the datasets, we observed that means were different, suggesting a difference in the distribution of features across the two datasets (see Appendix C). Even after normalization, the mean of the original dataset was significantly smaller than the new dataset. To investigate further, we visualized the input distributions of both datasets and as can be seen in the graph below, they are different. The peaks of the original dataset (left) are tightly aligned and consistent, whilst the new dataset (right) appears to have multiple peaks which vary in location, indicating a covariate shift.

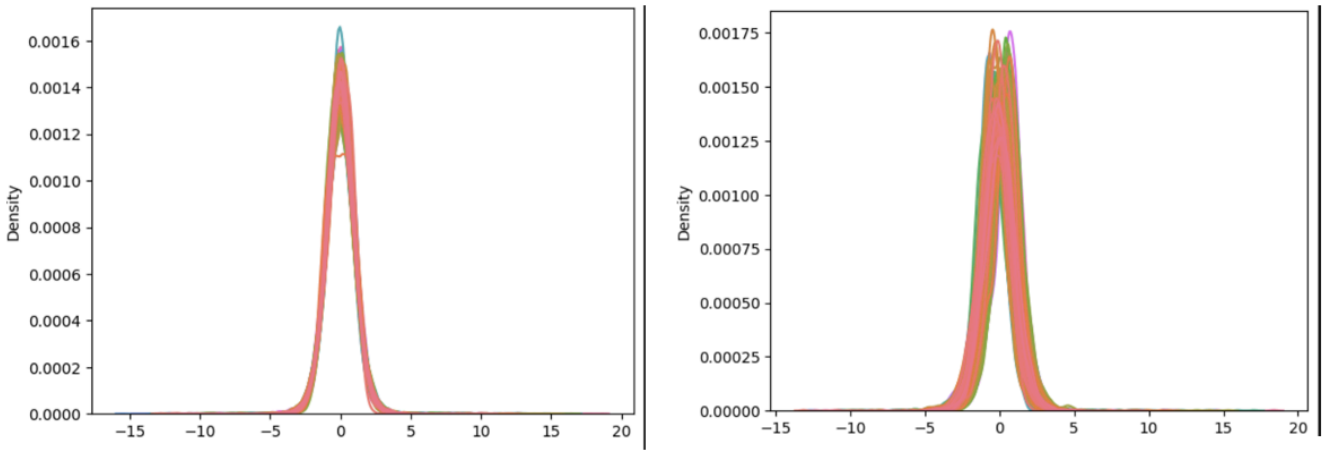


Figure 2: Plot showing the input density distribution of the original (left) and new (right) datasets

We employed density ratio estimation to address the covariate shift. This method estimates the ratio between the original and the new probability distribution, allowing us to reweight the training samples so that they better match the distribution of the test dataset. Initially, we attempted to incorporate the density ratio into our ensemble model. However, due to the complexity of our approach, integrating the reweighting mechanism across the base models proved to be challenging and despite multiple attempts, we were unsuccessful. As an alternative, we tested our base models - Logistic Regression, XGBoost and the RandomForest - with the density ratio estimation, finding that Logistic Regression produced the best result. We formulated our predictions for ‘Test Set 2’ using logistic regression.

For further improvement on the performance, we aim to integrate density ratio estimation into the final ensemble model. Given that the ensemble method consists of 3 base models, this will enable us to build upon all previous considerations of feature correlations, relationships, and key characteristics of the dataset. This will ensure better model performance when faced with data distribution shifts. Additionally, using the stacked ensemble method will help mitigate the effect of class imbalance, as our ensemble model performs better than logistic regression in this regard.

Conclusion

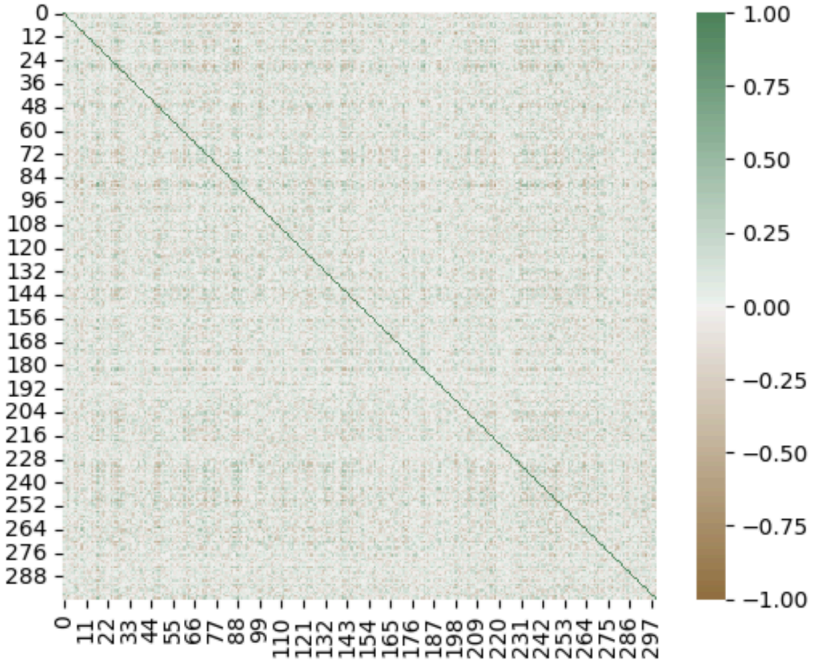
We integrated strong feature selection using a combination of XGBoost, Permutation Importance with Logistic Regression and Random Forest, resulting in 105 key features. These features were incorporated into a three-way stacked ensemble method. By using stratified 3-fold cross validation and hyperparameter optimisation, we developed a classification model. We utilised Weighted Log Loss as our primary evaluation metric to prioritise the minority class, as the dataset was heavily imbalanced. Our final model balanced generalisation and performance but could be improved to handle data distribution shifts.

Reference List

Bothma J, 2024, Seaborn Heatmaps: A guide to Data Visualization, DataCamp, viewed 18th April 2025, <https://www.datacamp.com/tutorial/seaborn-heatmaps>.

Data Professor 2022, How to handle imbalanced datasets in Python, online video, accessed 13th April 2025, <https://www.youtube.com/watch?v=4SivdTLIwHc>.
DataRobot 2025, Optimization metrics, accessed 15th April 2025, <https://docs.datarobot.com/en/docs/modeling/reference/model-detail/opt-metric.html>.
GeeksforGeeks 2024, What is Cross-Entropy Loss Function?, accessed 11th April 2025, <https://www.geeksforgeeks.org/what-is-cross-entropy-loss-function/>.
GeeksforGeeks 2025, F1 Score in Machine Learning, accessed 12th April 2025, <https://www.geeksforgeeks.org/f1-score-in-machine-learning/>.
Huyen, C 2022, Data Distribution Shifts and Monitoring, accessed 20th April 2025, <https://huyenchip.com/2022/02/07/data-distribution-shifts-and-monitoring.html#handling-data-shifts>.
Kavlakgilu, E & Russi, E 2024, What is XGBoost?, IBM, viewed 15th April 2025, <https://www.ibm.com/think/topics/xgboost#:~:text=XGBoost%20(eXtreme%20Gradient%20Boosting)%20is,scale%20well%20with%20large%20datasets>.
Li, H & Li, Z 2022, 'Text Classification Based on Machine Learning and Natural Language Processing Algorithms', Wireless Communications and Mobile Computing, accessed 9th April 2025, <https://onlinelibrary.wiley.com/doi/10.1155/2022/3915491>.
Lianne and Justin 2022, Handling Imbalanced Data in machine learning classification (Python) - 2, online video, accessed 13th April 2025, <https://www.youtube.com/watch?v=Bt5g7c2s38M>.
Mannemala, A 2021, 'Categorizing Customer feedback using Machine Learning', Iowa State University, accessed 12th April 2025, <https://dr.lib.iastate.edu/server/api/core/bitstreams/a67d53f7-15ea-4324-9138-7c1b7926a594/content>.
Mrsarthakgupta 2021, How to correct covariate shift?, accessed 21st April 2025, <https://medium.com/analytics-vidhya/how-to-correct-covariate-shift-76f9513b5883>.
Murel, J & Kavlakoglu, E 2024, What is transfer learning?, accessed 17th April 2025, <https://www.ibm.com/think/topics/transfer-learning>.
Noori, B 2021, 'Classification of Customer Reviews Using Machine Learning Algorithms', Applied Artificial Intelligence, vol.35, no.8, accessed 9th April 2025, <https://www.tandfonline.com/doi/full/10.1080/08839514.2021.1922843>.
Rezaei-Dastjerdehei, M, Mijani, A & Fatemizadeh, E 2020, 'Addressing Imbalance in Multi-Label Classification Using Weighted Cross Entropy Loss Function', 27th National and 5th International Iranian Conference on Biomedical Engineering, viewed 11th April 2025, <https://ieeexplore.ieee.org/document/9319440>.
Sarantitis, G 2020, Data Shift in Machine Learning: what is it and how to detect it, viewed 20th April 2025, <https://gsarantitis.wordpress.com/2020/04/16/data-shift-in-machine-learning-what-is-it-and-how-to-detect-it/>.
Stryker, C & Holdsworth, J 2024, What is NLP (natural language processing)?, IBM, viewed 19th April 2025, <https://www.ibm.com/think/topics/natural-language-processing>.

Appendix A
Heatmap



Heatmap displaying the correlation between all of the features. There is no strong positive or negative correlation between any of the different features.

Top 20 correlation pair values

Top 20 most correlated feature pairs (by absolute value)

	Feature 1	Feature 2	Correlation
0	17	87	0.687621
1	88	218	-0.679775
2	27	88	-0.656537
3	76	249	0.651349
4	17	218	-0.629098
5	72	218	-0.625474
6	88	249	0.610552
7	27	249	-0.605289
8	124	172	0.596812
9	111	249	-0.596312
10	79	268	0.594595
11	48	263	0.594304
12	17	88	0.593439
13	46	249	-0.582562
14	86	268	-0.579688
15	76	88	0.578680
16	72	88	0.574501
17	27	218	0.569282
18	27	182	0.568703
19	218	249	-0.568073

Table showing the top 20 most correlated feature pairs (by absolute value) in the dataset

Appendix B

Important feature selections (Ranking)

Model	Important Features top 50
Random Forest	{131, 259, 132, 265, 268, 140, 270, 14, 17, 274, 148, 279, 283, 157, 287, 292, 293, 39, 168, 44, 172, 46, 47, 53, 182, 55, 59, 71, 205, 79, 85, 86, 87, 216, 88, 218, 90, 215, 89, 222, 99, 228, 100, 231, 111, 240, 242, 249, 123, 124}
Logistic regression	{130, 2, 263, 135, 9, 265, 267, 268, 141, 270, 273, 19, 152, 282, 154, 284, 32, 289, 290, 164, 41, 172, 48, 177, 176, 187, 61, 64, 65, 66, 67, 198, 200, 201, 205, 206, 79, 215, 88, 216, 87, 95, 97, 228, 105, 106, 236, 112, 249, 127}
XGBoost	{257, 132, 134, 263, 265, 10, 268, 141, 270, 271, 144, 17, 274, 276, 20, 283, 157, 290, 162, 41, 172, 48, 50, 182, 55, 61, 64, 70, 71, 205, 82, 86, 215, 88, 216, 90, 218, 87, 222, 227, 99, 228, 230, 231, 111, 113, 116, 118, 249, 125}

Important Feature selection based on unions

Model	Feature Counts	Important Features top 50
Random Forest + Logistic Regression	88	[2, 9, 14, 17, 19, 32, 39, 41, 44, 46, 47, 48, 53, 55, 59, 61, 64, 65, 66, 67, 71, 79, 85, 86, 87, 88, 89, 90, 95, 97, 99, 100, 105, 106, 111, 112, 123, 124, 127, 130, 131, 132, 135, 140, 141, 148, 152, 154, 157, 164, 168, 172, 176, 177, 182, 187, 198, 200, 201, 205, 206, 215, 216, 218, 222, 228, 231, 236, 240,

		242, 249, 259, 263, 265, 267, 268, 270, 273, 274, 279, 282, 283, 284, 287, 289, 290, 292, 293]
XGBoost + Random Forest	50	[2, 9, 19, 32, 41, 48, 61, 64, 65, 66, 67, 79, 87, 88, 95, 97, 105, 106, 112, 127, 130, 135, 141, 152, 154, 164, 172, 176, 177, 187, 198, 200, 201, 205, 206, 215, 216, 228, 236, 249, 263, 265, 267, 268, 270, 273, 282, 284, 289, 290]
Logistic regression + XGBoost	50	[2, 9, 19, 32, 41, 48, 61, 64, 65, 66, 67, 79, 87, 88, 95, 97, 105, 106, 112, 127, 130, 135, 141, 152, 154, 164, 172, 176, 177, 187, 198, 200, 201, 205, 206, 215, 216, 228, 236, 249, 263, 265, 267, 268, 270, 273, 282, 284, 289, 290]
XGBoost + Random Forest + Logistic Regression	105	[2, 9, 10, 14, 17, 19, 20, 32, 39, 41, 44, 46, 47, 48, 50, 53, 55, 59, 61, 64, 65, 66, 67, 70, 71, 79, 82, 85, 86, 87, 88, 89, 90, 95, 97, 99, 100, 105, 106, 111, 112, 113, 116, 118, 123, 124, 125, 127, 130, 131, 132, 134, 135, 140, 141, 144, 148, 152, 154, 157, 162, 164, 168, 172, 176, 177, 182, 187, 198, 200, 201, 205, 206, 215, 216, 218, 222, 227, 228, 230, 231, 236, 240, 242, 249, 257, 259, 263, 265, 267, 268, 270, 271, 273, 274, 276, 279, 282, 283, 284, 287, 289, 290, 292, 293]

Appendix C

Summary Statistics of the original dataset (no standardisation)

	0	1	2	3	4	5	6	7	8	9	...
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	...
mean	-0.021449	-0.000912	0.012368	0.045447	0.034961	0.070815	0.074531	0.049599	0.027968	0.055083	...
std	0.986789	1.008141	1.009709	0.985909	0.988168	0.990837	0.975877	1.014643	0.996102	1.034924	...
min	-8.163800	-11.982000	-3.502000	-4.704000	-7.017800	-4.991400	-5.802900	-3.559200	-6.818100	-13.353000	...
25%	-0.629273	-0.601035	-0.652678	-0.567387	-0.589565	-0.570862	-0.488445	-0.636760	-0.559490	-0.496065	...
50%	-0.026099	0.026874	-0.042700	0.069475	0.026490	0.071909	0.098494	0.014708	0.050312	0.143520	...
75%	0.581712	0.627327	0.620073	0.677163	0.639463	0.704340	0.666788	0.691455	0.647860	0.745035	...
max	4.575000	4.232200	6.586800	3.867900	7.705200	5.112700	10.495000	4.853900	4.515200	2.694500	...

8 rows × 300 columns

Summary Statistics of the new dataset (no standardisation)

	0	1	2	3	4	5	6	7	8	9	...
count	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	...
mean	0.072500	-0.024150	-0.058576	-0.128789	-0.125521	-0.295223	-0.282753	-0.160675	-0.091492	-0.209856	...
std	1.088832	1.015746	0.992920	1.061684	1.027777	1.005629	1.092321	0.920479	1.013461	0.898514	...
min	-5.017100	-11.223000	-2.995200	-4.082500	-4.370400	-3.928000	-5.670600	-3.444500	-4.325600	-13.554000	...
25%	-0.599570	-0.665882	-0.724463	-0.834947	-0.761053	-0.924702	-0.973285	-0.765850	-0.729328	-0.691520	...
50%	0.081227	-0.045906	-0.078369	-0.102365	-0.115950	-0.308385	-0.245875	-0.147505	-0.075513	-0.154855	...
75%	0.754023	0.591692	0.542482	0.543772	0.532195	0.305893	0.441943	0.447242	0.576505	0.360278	...
max	4.864500	5.389600	4.118000	3.646000	3.619600	5.575000	9.010800	3.524600	3.912300	2.190400	...

Summary Statistics of the original dataset (standardisation)

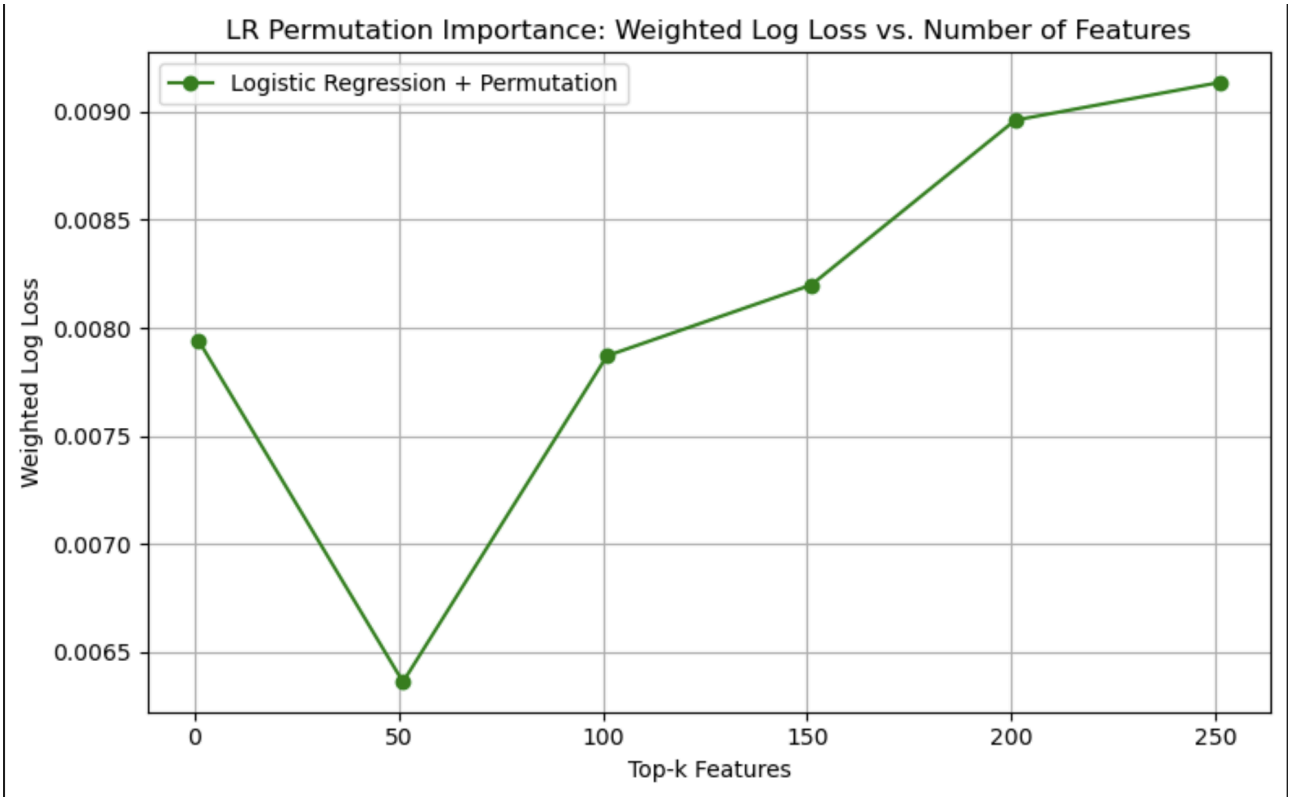
	0	1	2	3	4	5	6	7	8	
count	1.000000e+04	1.000000e+04	1.000000e+04	1.000000e+04	1.000000e+04	1.000000e+04	1.000000e+04	1.000000e+04	1.000000e+04	1.000000e+
mean	4.973799e-18	8.526513e-18	7.993606e-18	-9.947598e-18	9.947598e-18	1.065814e-17	1.847411e-17	-1.563194e-17	2.415845e-17	-3.552714e-
std	1.000050e+00	1.000050e+00	1.000050e+00	1.000050e+00	1.000050e+00	1.000050e+00	1.000050e+00	1.000050e+00	1.000050e+00	1.000050e+
min	-8.251775e+00	-1.188493e+01	-3.480748e+00	-4.817568e+00	-7.137569e+00	-5.109282e+00	-6.023016e+00	-3.556895e+00	-6.873202e+00	-1.295627e-
25%	-6.159916e-01	-5.953069e-01	-6.586832e-01	-6.216242e-01	-6.320360e-01	-6.476432e-01	-5.769210e-01	-6.764873e-01	-5.897863e-01	-5.32576
50%	-4.711598e-03	2.756232e-02	-5.454050e-02	2.437331e-02	-8.573086e-03	1.104665e-03	2.455700e-02	-3.438957e-02	2.243260e-02	8.545660e-
75%	6.112677e-01	6.231973e-01	6.018911e-01	6.407763e-01	6.117703e-01	6.394158e-01	6.069267e-01	6.326246e-01	6.223490e-01	6.667025e
max	4.658221e+00	4.199139e+00	6.511538e+00	3.877278e+00	7.762471e+00	5.088764e+00	1.067858e+01	4.735203e+00	4.505017e+00	2.550477e+

Summary Statistics of the new dataset (standardisation)

	0	1	2	3	4	5	6	7	8	9
count	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000	2020.000000
mean	0.095212	-0.023052	-0.070265	-0.176735	-0.162412	-0.369441	-0.366134	-0.207249	-0.119934	-0.256012
std	1.103465	1.007594	0.983421	1.076912	1.040136	1.014979	1.119378	0.907240	1.017478	0.868237
min	-5.062787	-11.132024	-2.978796	-4.187154	-4.458335	-4.035994	-5.887438	-3.443845	-4.370823	-13.150500
25%	-0.585890	-0.659634	-0.729781	-0.893022	-0.805586	-1.004773	-1.073771	-0.803721	-0.760297	-0.721445
50%	0.104056	-0.044633	-0.089869	-0.149932	-0.152726	-0.382725	-0.328343	-0.194269	-0.103891	-0.202864
75%	0.785893	0.587848	0.525043	0.505473	0.503213	0.237264	0.376512	0.391924	0.550711	0.294910
max	4.951611	5.347250	4.066356	3.652196	3.627743	5.555362	9.157621	3.425022	3.899727	2.063363

8 rows × 300 columns

Appendix D



Graph showing that 50 is the best number of features found by the LR Permutation Importance